

Welcome to

11. Network Attacks

EK Software Security

Henrik Kramselund he/him han/ham hlk@zencurity.com @kramse

Slides are available as PDF, https://codeberg.org/kramse

11-network-attacks.tex in the repo security-courses

© license CC BY 3.0. 2026 Henrik Kramselund

Plan for today



Subjects

- Auditing Application Protocols
- Example protocols and vulnerabilities
- Abstract Syntax Notation (ASN.1) problems
- Domain Name System (DNS) problems
- Firewalls and related issues
- Network Security Monitoring

Exercises

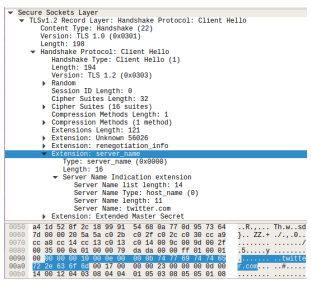
- Nmap and SYN flooding exercises

Reading Summary



Browse: *TCP Synfloods - an old yet current problem, and improving pf's response to it* Henning Brauer, BSDCan 2017 http://quigon.bsws.de/papers/2017/bsdcan/

Goals: Introduction to Auditing Application Protocols



Often you dont need to audit the whole protocol in detail

Sometimes people can't tell which protocols, ports and services they use ...

And you need to configure a firewall/network filter

Picture: Wireshark with TLS SNI, recent Exim CVE-2019-15846 was SNI parsing

Plus SYN flooding, and Denial of Service

What is pentesting and a pentest



> A penetration test, colloquially known as a pen test, pentest or ethical hacking, is an authorized simulated cyberattack on a computer system, performed to evaluate the security of the system;[1][2] this is not to be confused with a vulnerability assessment.[3] The test is performed to identify weaknesses (also referred to as vulnerabilities), including the potential for unauthorized parties to gain access to the system's features and data,[4][5] as well as strengths,[6] enabling a full risk assessment to be completed.

Source: quote from https://en.wikipedia.org/wiki/Penetration_test

Penetration testing is a simulation, with good intentions

People around the world constantly *test your defenses*

Often better to test at planned times

Time schedule



Plan for the day

12:45 - 13:45 – Session 0: Teaching Network Attacks intro

13:45 - 14:00 – 15min break

14:00 - 14:45 – Session 1: Exercise: *sshole

14:45 - 15:00 – 15min break

15:00 - 16:45 – Sessions 2+3: Teaching and exercises in Networks and DDoS

Session 0: Teaching and exercise: Network Attacks intro





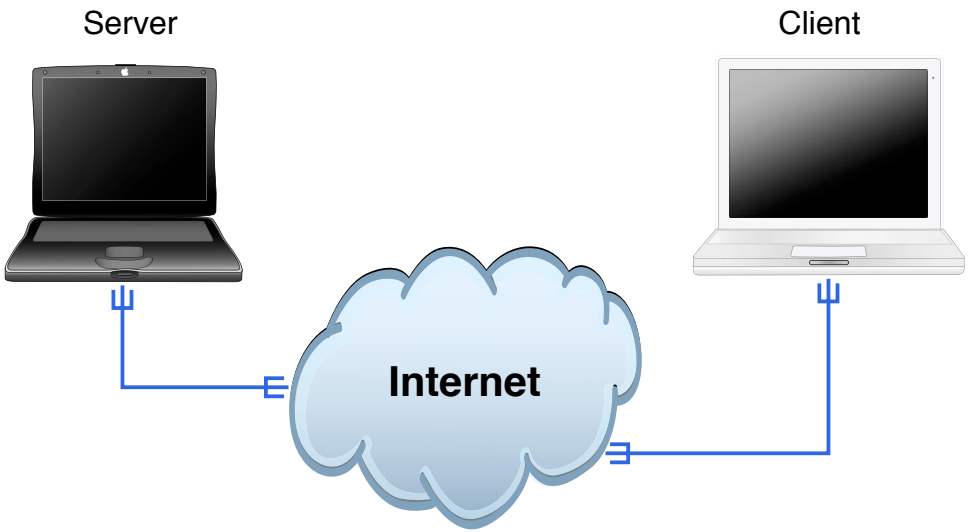
Computer network

In computer science, computer engineering, and telecommunications, **a network** is a **group** of communicating computers and peripherals called hosts, which communicate data to other hosts via communication protocols, as facilitated by networking hardware.

Within a computer network, **hosts** are identified by **network addresses**, which allow networking hardware to locate and identify hosts. Hosts may also have **hostnames**, memorable labels for the host nodes, which can be mapped to a network address using a hosts file or a name server such as **Domain Name Service**. The physical medium that supports information exchange includes **wired media like copper cables, optical fibers, and wireless radio-frequency media**. The arrangement of hosts and hardware within a network architecture is called the **network topology**.[1][2]

Source: https://en.wikipedia.org/wiki/Computer_network

Internet Today



Clients and servers, roots in the academic world

Protocols are old, very old! RFC 791: STD 5: Internet Protocol is from September 1981!

Very few protocols were encrypted, today a lot has switched to HTTPS and TLS

Internet is Open Standards!



We reject kings, presidents, and voting.
We believe in rough consensus and running code.
– The IETF credo Dave Clark, 1992.

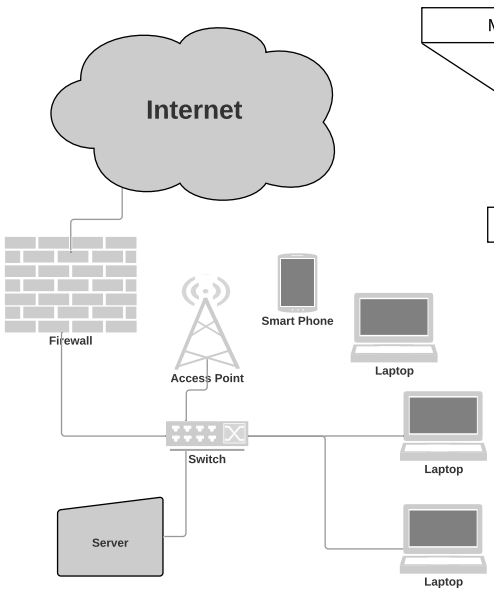
Request for comments (RFC) – a series of documents spanning decades

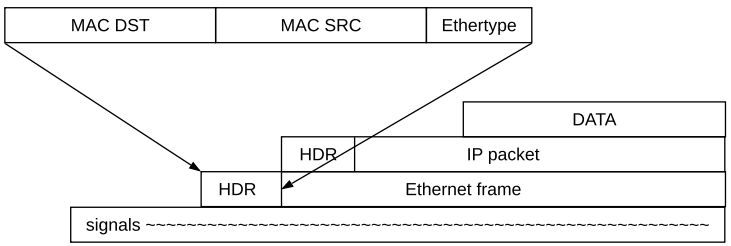
RFC, BCP, FYI, informational – first ones from 1969!

Are not updated but status is changed to Obsoleted when new versions are published

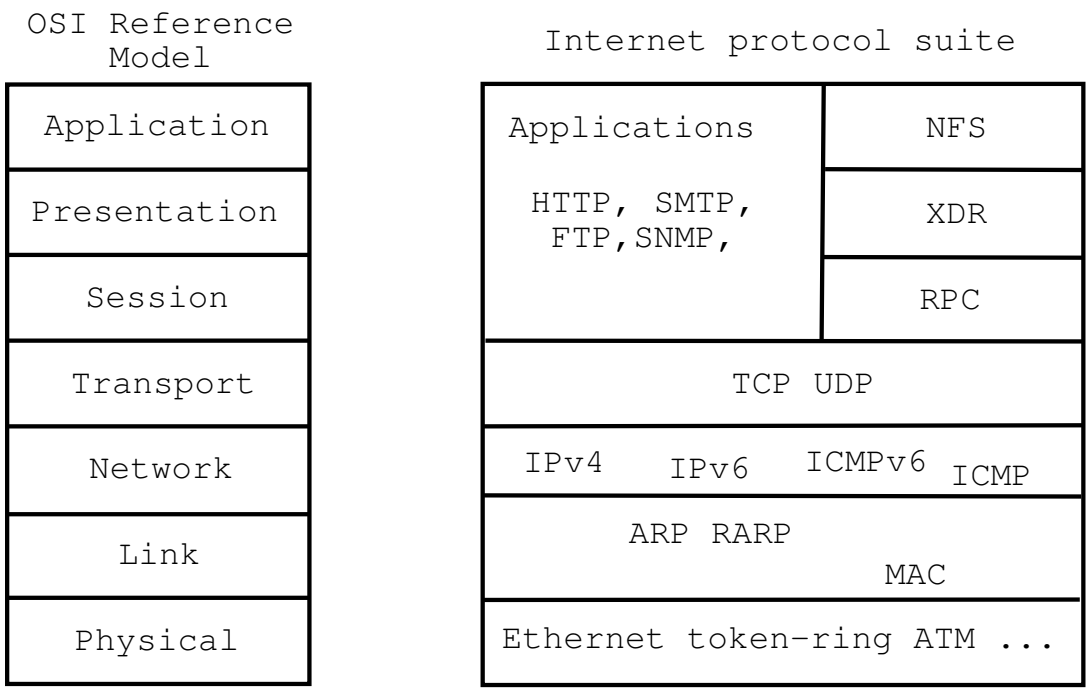
Standards track:
Proposed Standard $\rightarrow$ Draft Standard $\rightarrow$ Standard

Network data at different layers




MAC DST
MAC SRC
Ethertype
DATA
HDR
IP packet
HDR
Ethernet frame
signals ~~~

Protocols: OSI and Internet models



Protocols are rules, like how to say; Hello! Don't worry too much yet, this is an overview!

Recommended network technologies to learn



Small set of recommended networking technologies to learn

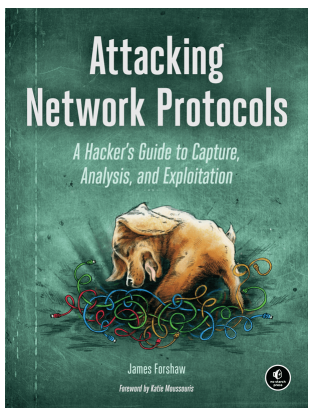
Networking: Basic Protocols from the Internet Protocols suite – TCP/IP

- Network Layer: Ethernet, Address Resolution Protocol (ARP), IPv4 and ICMPv4
- Transport Layer: Transmission Control Protocol (TCP) and User Datagram Protocol (UDP)
- Common upper layer: Dynamic Host Configuration Protocol (DHCP), Domain Name System (DNS), Hypertext Transfer Protocol (HTTP)

- Later add the encrypted/secure versions like Hypertext Transfer Protocol Secure (HTTPS) which uses Transport Layer Security (TLS)
- Later add Wi-Fi and IPv6/ICMPv6 with Neighbor Discovery Protocol (NDP)

Enough to be able to go to other resources, and a connected path from ARP to TLS

Reversing and Attacking Network Protocols



A method with lots detail can be found in the book,
Attacking Network Protocols A Hacker's Guide to Capture, Analysis, and Exploitation
by James Forshaw December 2017, 336 pp. ISBN-13: 9781593277505

https://nostarch.com/networkprotocols

Auditing Application Protocols



- Collect documentation
- Identify Elements of Unknown Protocols
- Use packet sniffers, tcpdump and Wireshark
- Initiate the Connection Several Times
- Replay traffic, can sometimes replay even encrypted traffic, see wireless WEP attacks

Note: We investigate protocols, so we can see what is sent, so we can design *payloads* which create problems for implementations - applications

Reverse Engineer Applications



```
(gdb) disas main
Dump of assembler code for function main:
   0x0000000000000580 <+0>: lea    0x1ed(%rip),%rdi        # 0x774
   0x0000000000000587 <+7>: sub    $0x8,%rsp
   0x000000000000058b <+11>: mov    $0x7fff,%esi
   0x0000000000000590 <+16>: xor    %eax,%eax
   0x0000000000000592 <+18>: callq  0x560 <printf@plt>
   0x0000000000000597 <+23>: lea    0x1ed(%rip),%rdi        # 0x78b
   0x000000000000059e <+30>: mov    $0xffff8000,%esi
   0x00000000000005a3 <+35>: xor    %eax,%eax
   0x00000000000005a5 <+37>: callq  0x560 <printf@plt>
   0x00000000000005aa <+42>: xor    %eax,%eax
   0x00000000000005ac <+44>: add    $0x8,%rsp
   0x00000000000005b0 <+48>: retq
End of assembler dump.
```

- It is possible to debug, disassemble and reverse engineer applications
- Calling socket functions, seeing structs, data types etc.
- Examine strings: HTTP, FTP, SMTP etc. all uses semi-english words GET, EHLO, PASS

Special values



- Examine special values
- What are the defined/used values
- What happens if this is changed? Do they cover values outside of the used ranges? Case/switch constructs
- Use trace functions in the operating system, can capture, analyze and replay sometimes

Buffer Overflow when receiving



- When you see data enter the application, identify functions
- Consider if they use dangerous functions, strcpy and friends
- How much space is available, allocated etc.
- Basic stuff and similar across applications

- Repeat everything we learned about string processing, integeroverflows/underflows etc. Just from the network
- Often trying to abuse will lead to denial of service

- If some rock solid service starts bouncing down and up, maybe look into traffic received.
- This is what honeypots also do

Binary Protocols



- Some protocols use binary formats
- Example DNS, which is a complex protocol
- When parsing DNS use standard libraries!
- When attacking DNS applications, use standard libraries! ☺
- DNS is just an example, new protocols may not be implemented in easy to use tools – but someone might have analyzed it or parts already!

ASN.1 problems



- Abstract format designed for representing objects in a machine independent format
- Used for various technologies in use on the internet:
- Certificates and key encoding
- Simple Network Management Protocol (SNMP)
- ISAKMP part of IPsec
- Lightweight Directory Access Protocol (LDAP)

Linux Kernel ASN.1



- CVE-2016-0758 Integer overflow in lib/asn1_decoder.c in the Linux kernel before 4.6 allows local users to gain privileges via crafted ASN.1 data.
  https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2016-0758
- Linux kernel have about 5 ASN.1 parsers
  https://www.x41-dsec.de/de/lab/blog/kernel_userspace/

ASN.1 parsing CVEs – 2003 to 2019



- CVE-2003-0818 Microsoft MSASN1.DLL (MS04-007), CVSS 9.8: BER length integer overflows, RCE via Kerberos, NTLM, IPsec, TLS
  https://docs.microsoft.com/en-us/security-updates/securitybulletins/2004/ms04-007
- CVE-2016-2053 Linux kernel, CVSS 4.7: panic in `asn1_ber_decoder`
  https://github.com/torvalds/linux/commit/0d62e9dd6da45bbf0f33a8617afc5fe774c8f45f
- CVE-2016-2108 OpenSSL, CVSS 9.8: "negative zero", memory corruption via crafted ANY field
  https://nvd.nist.gov/vuln/detail/CVE-2016-2108
- CVE-2016-5080 Objective Systems ASN1C (Cisco ASR 5000), CVSS 9.8: integer overflow in compiler-generated code
  https://www.cisco.com/c/en/us/support/docs/csa/cisco-sa-20160721-asn1c.html
- CVE-2019-9162 Linux kernel, CVSS 7.8: missing ASN.1 length checks in SNMP NAT helper, OOB read/write
  https://bugs.chromium.org/p/project-zero/issues/detail?id=1776

Need a rabbit hole, check out ASN.1 parsing, parsing anything is hard!

ASN.1 parsing CVEs – 2021 to 2025



- CVE-2021-43527 Mozilla NSS "BigSig", CVSS 9.8: heap overflow on oversized DER-encoded signature https://nvd.nist.gov/vuln/detail/CVE-2021-43527
- CVE-2022-44640 Heimdal Kerberos KDC, CVSS 9.8: invalid free on crafted DER CHOICE, generated by its ASN.1 compiler https://github.com/heimdal/heimdal/security/advisories/GHSA-88pm-hfmq-7vv4
- CVE-2023-0286 OpenSSL, CVSS 7.4: X.400 address type confusion in GeneralName, attacker pointers to `memcmp` https://www.openssl.org/news/secadv/20230207.txt
- CVE-2024-12133 libtasn1, CVSS 5.3: many SEQUENCE OF/SET OF elements, quadratic decoding, DoS https://gitlab.com/gnutls/libtasn1/-/issues/52
- CVE-2025-0343 Swift ASN.1, CVSS 7.5: primitive vs. constructed form not enforced, crash on crafted BER/DER https://nvd.nist.gov/vuln/detail/CVE-2025-0343
- CVE-2025-13151 libtasn1, CVSS 7.5 (upstream: Low): one-byte stack overflow in `asn1_expand_octet_string` https://gitlab.com/gnutls/libtasn1/-/issues/55

Same bug class for 20+ years: trusting length, type and form fields

Type Length Value TLVs



> TLV sequences are easily searched using generalized parsing functions; New message elements which are received at an older node can be safely skipped and the rest of the message can be parsed. This is similar to the way that unknown XML tags can be safely skipped; TLV elements can be placed in any order inside the message body; TLV elements are typically used in a binary format which makes parsing faster and the data smaller than in comparable text based protocols.

Source: https://en.wikipedia.org/wiki/Type-length-value

- Type Length Value is an encoding used in data communication
- For example in Link Layer Discovery Protocol (LLDP), SSH, RADIUS

Network Authentication



IPMI Authentication Bypass via Cipher 0

Dan Farmer identified a serious failing of the IPMI 2.0 specification, namely that cipher type 0, an indicator that the client wants to use clear-text authentication, actually allows access with any password. Cipher 0 issues were identified in HP, Dell, and Supermicro BMCs, with the issue likely encompassing all IPMI 2.0 implementations. It is easy to identify systems that have cipher 0 enabled using the `ipmi_cipher_zero` module in the Metasploit Framework.

- Sometimes people add network functionality to existing applications – and do this badly
- We have seen applications like IPMI and others
- Is this a network problem, server problem, hardware problem? – **discuss**

Source: https://blog.rapid7.com/2013/07/02/a-penetration-testers-guide-to-ipmi/

Even securiy protocols and software have problems



- ISAKMP example
- IKE(v1) has been critized as being overly complex
- Needed bake-off sessions where vendors meet and tried negotiating
- Searching for CVE ISAKMP show multiple vulnerabilities in various implementations, including firewalls and tcpdump
- Source: AoSSA chapter 16: Network Application Protocols
  The Art of Software Security Assessment Identifying and Preventing Software Vulnerabilities Mark Dowd, John McDonald, Justin Schuh ISBN: 9780321444424

Break



Photo by Alexas Fotos on Unsplash

Session 1: Exercise: *sshole



password: AAAAAAAAAAAAAAAA...AAAAAAAAAAAAAA



*sshole

so openssh on musl linux (alpine, openwrt, buildroot, yocto, and a lot of the docker images you've pulled) has had a little problem since 2016.

accounts that should never take a password at all (root, your key-only deploy user, ldap users, the default user on alpine's aws and azure images) take any password longer than 256 characters.

yes, any password. yes. i know. yes.

Source: https://sshhole.com/!

Lets look into this including CVE-2016-6210 and CVE-2019-5021

Release notes: OpenSSH 10.6p2 (2026-11-05)



New version

```
Portable OpenSSH 10.6p2 was released on 2026-11-05. It will be available
from the mirrors listed at https://www.openssh.com/.
...
Security
========
 * sshd(8): avoid password authentication bypass that is exploitable
   through sshd(8) that affects some Linux systems when built and run
   with PAM support disabled. We consider the root cause of this bug
   to be outside sshd, but it is exploitable via sshd and can be
   avoided there.

   This bug was reported by Özgün Kültekin of Wakefield
   (wakefield.co)
```

Source: https://www.openssh.org/releasenotes.html bold by me

OpenSSH receives large number of security bug reports



```
Recently the OpenSSH team have received a large number of security
bug reports, many of which are findings from AI models or made with
AI assistance. While many AI reports are determined not to have
security impact when considered in the context of a realistic
threat model, we very much welcome these reports, especially when
combined with human triage, analysis, test-cases and particularly
when accompanied by proposed fixes.
```

Source: https://www.openssh.org/releasenotes.html

OpenSSH AI changes disclosure policies!



```
** We have seen a number of cases where a security bug identified
** by AI tools is subsequently independently discovered by a
** different researcher. This suggests that adversaries who do not
** report bugs to OSS projects are likely to be able to discover
** these bugs too. Given this, the OpenSSH team will, for now, be
** making more frequent releases to get bugfixes into users' hands
** more quickly rather than batching them until the next planned
** release.
```

Source: https://www.openssh.org/releasenotes.html

Saw something similar about cURL!

Lets try it!



- Read up on the bugs – multiple, note how it would have been hard predicting the interactions!
- Lets find the container images – run them in your K8s clusters
- Maybe it needs some configuration?
- When the attack works, retry with updated software/images – and same configuration

Bonus question: would any of our previous Defense in Depth strategies work? Running container as non-root?

Would VictoriaLogs or similar work? What would that provide us?

Break



Photo by Alexas Fotos on Unsplash

Sessions 2+3: Teaching and exercises in Networks and DDoS



Networking with some pentesting



What is pentest

> A penetration test, informally pen test, is an attack on a computer system that looks for security weaknesses, potentially gaining access to the computer's features and data.[1][2]

Source: quote from https://en.wikipedia.org/wiki/Penetration_test

Penetration testing is a simulation, with good intentions

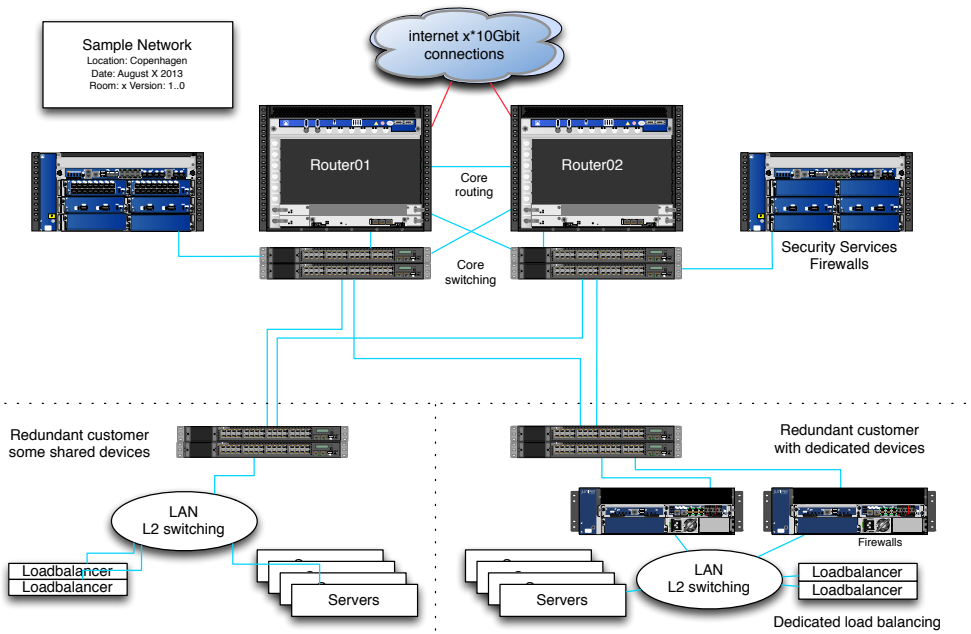
People around the world constantly *test your defenses*

Often better to test at planned times

How to create DDoS simulations, tools and process

I use and recommend Kali Linux as the base for this

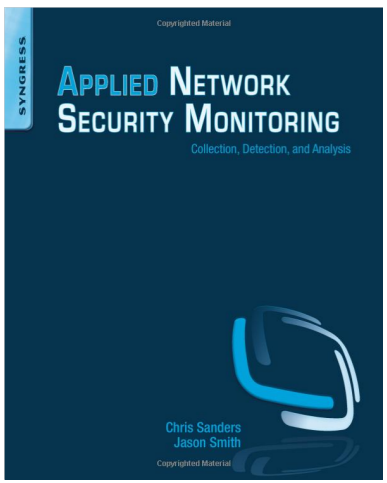
Networks today



Conclusion: networks are complex

Many places where you can block, filter, restrict, limit, avoid, process, change, observe

Book: Applied Network Security Monitoring (ANSM)

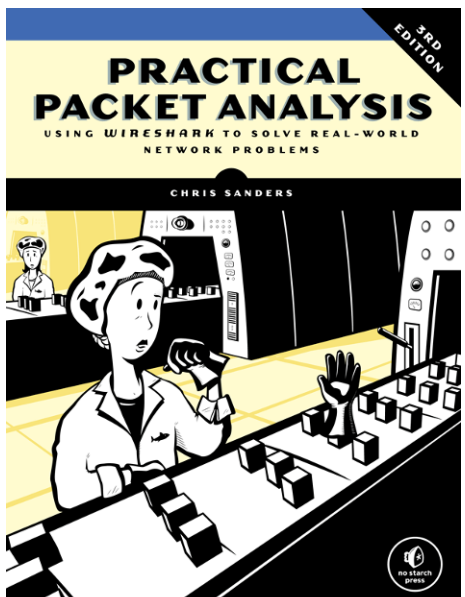


Applied Network Security Monitoring: Collection, Detection, and Analysis 1st Edition

Chris Sanders, Jason Smith eBook ISBN: 9780124172166 Paperback ISBN: 9780124172081 496 pp. Imprint: Syngress, December 2013

https://www.elsevier.com/books/applied-network-security-monitoring/unknown/978-0-12-417208-1

Book: Practical Packet Analysis (PPA)



Practical Packet Analysis, Using Wireshark to Solve Real-World Network Problems by Chris Sanders, 3rd Edition April 2017, 368 pp. ISBN-13: 978-1-59327-802-1

https://nostarch.com/packetanalysis3

Internet is Open Standards!



> We reject kings, presidents, and voting.
> We believe in rough consensus and running code.
> – The IETF credo Dave Clark, 1992.

Request for comments - RFC - er en serie af dokumenter

RFC, BCP, FYI, informational
de første stammer tilbage fra 1969

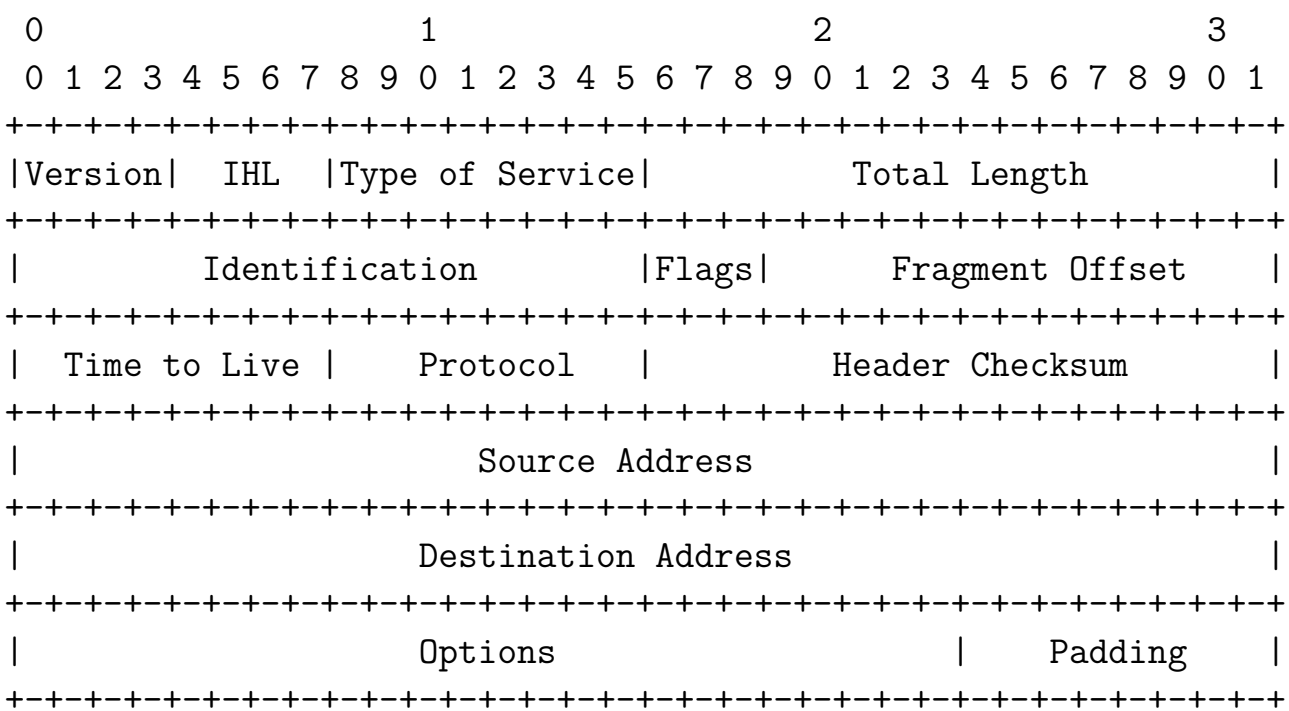
Ændres ikke, men får status Obsoleted når der udkommer en nyere version af en standard

Standards track:
Proposed Standard $\rightarrow$ Draft Standard $\rightarrow$ Standard

Åbne standarder = åbenhed, ikke garanti for sikkerhed

IPv4 packets – header - RFC-791





Example Internet Datagram Header

The basic tools for looking at packets

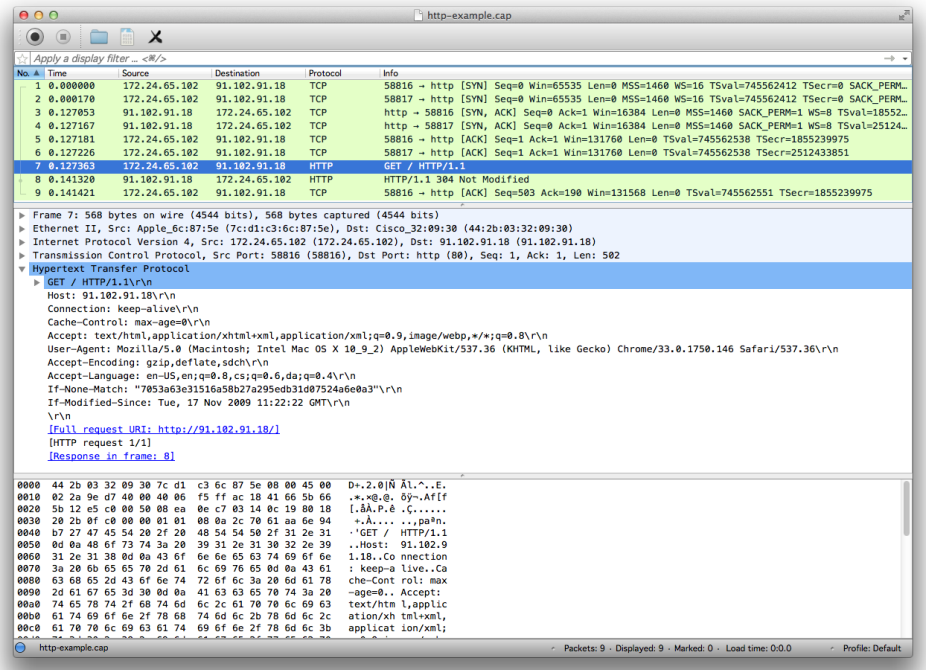


Knowledge and insight

- Networks have end-points and conversations on multiple layers
- Name resolution includes low level MAC addresses, and IP – names
- Tcpdump format, built-in to many network devices
- Wireshark is advanced, try right-clicking different places
- Remote packet dumps, like `tcpdump -i eth0 -w packets.pcap`
- Story: tcpdump was originally written in 1988 by Van Jacobson, Sally Floyd, Vern Paxson and Steven McCanne who were, at the time, working in the Lawrence Berkeley Laboratory Network Research Group https://en.wikipedia.org/wiki/Tcpdump

You can learn a lot by just listening!

Wireshark - graphical network sniffer



Capture - Options, select a network interface

http://www.wireshark.org

DNS problems



> The Domain Name System (DNS) [32][33] provides for a distributed database mapping host names to IP addresses. An intruder who interferes with the proper operation of the DNS can mount a variety of attacks, including denial of service and password collection. There are a number of vulnerabilities.

Source: *Security Problems in the TCP/IP Protocol Suite*, S.M. Bellovin 1989
https://www.cs.columbia.edu/~smb/papers/ipext.pdf

We have a lot of the same problems in DNS today

Plus some more caused by middle-boxes, NAT, DNS size, DNS inspection

- DNS must allow both UDP and TCP port 53
- Your DNS servers must have updated software, which use updated protocol behaviour
  see DNS flag day https://dnsflagday.net/
- DNS is unencrypted

Recommended Software – Unbound



Unbound is a validating, recursive, caching DNS resolver. It is designed to be fast and lean and incorporates modern features based on open standards.

To help increase online privacy, Unbound supports DNS-over-TLS which allows clients to encrypt their communication. In addition, it supports various modern standards that limit the amount of data exchanged with authoritative servers.

https://www.nlnetlabs.nl/projects/unbound/about/

My preferred local DNS server

DNS problems



- Failure to Deal with Invalid Label Lengths
- Insufficient Destination Lengths Check
- Insufficient Source Length Checks
- Pointer Values Not Verified In Packet
- Special Pointer Values
- Length Variables
- These are from the book: AoSSA chapter 16: Network Application Protocols
  The Art of Software Security Assessment Identifying and Preventing Software Vulnerabilities Mark Dowd, John McDonald, Justin Schuh ISBN: 9780321444424

Labels and pointers within packets save bytes, but make it more complex!

Firewalls and related issues



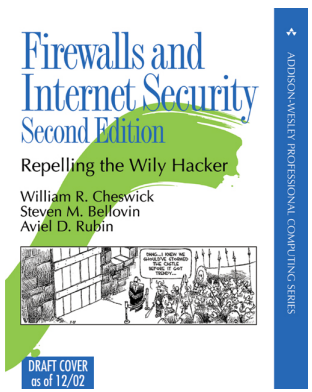
> In computing, a firewall is a network security system that monitors and controls incoming and outgoing network traffic based on predetermined security rules.[1] A firewall typically establishes a barrier between a trusted internal network and untrusted external network, such as the Internet.[2]

Source: Wikipedia

- https://en.wikipedia.org/wiki/Firewall_(computing)
- http://www.wilyhacker.com/ Cheswick chapter 2 PDF *A Security Review of Protocols: Lower Layers*
- Network layer, packet filters, application level, stateless, stateful

Firewalls are by design a choke point, natural place
to do network security monitoring!

Back in the day: Firewalls and Internet Security, 1994



- *Firewalls and Internet Security: Repelling the Wily Hacker* , Second Edition 2003, William R. Cheswick, Steven M. Bellovin, and Aviel D. Rubin, **2003** http://www.wilyhacker.com/
- The full PDF—and the full LaTeX source of the book. Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
- How to configure firewalls often boil down to, should we allow protocol X
- If we allow certain protocols through a firewall, we are asking for trouble

Definition of firewalls – multiple definitions exist



We define a firewall as **a collection of components** placed between two networks that collectively have the following properties:

- All traffic from inside to outside, and vice-versa, must pass through the firewall.
- Only authorized traffic, as defined by the **local security policy**, will be allowed to pass.
- The firewall itself is immune to penetration.

We should note that these are design goals; a failure in one aspect does not mean that the collection is not a firewall, simply that it is not a very good one.

We will consider this a firewall, but we know today that **both inside and outside at meaningless**, since we have **multiple networks inside, we have partner network connections etc.**

Source: **Cheswick94** by **Cheswick94 Cheswick94**

Definition of firewalls – Wikipedia



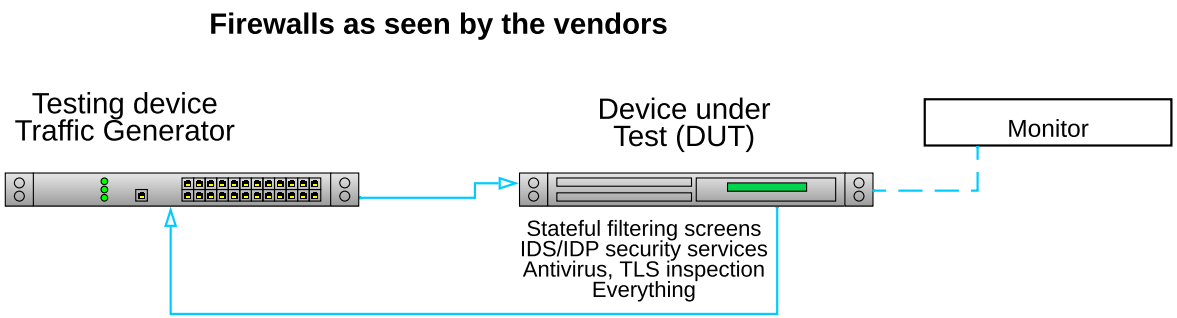
Another short definition that encapsulates this is found on Wikipedia, and may suffice in many situations. Again there will typically be multiple networks, zones or areas of the networks with varying degrees of trust.

> In computing, a firewall is a **network security system that monitors and controls incoming and outgoing network traffic based on predetermined security rules**.[1] A firewall typically establishes a barrier between a trusted network and an untrusted network, such as the Internet.[2]

Source: Wikipedia https://en.wikipedia.org/wiki/Firewall_(computing)

TL;DR Not necessarily a single device

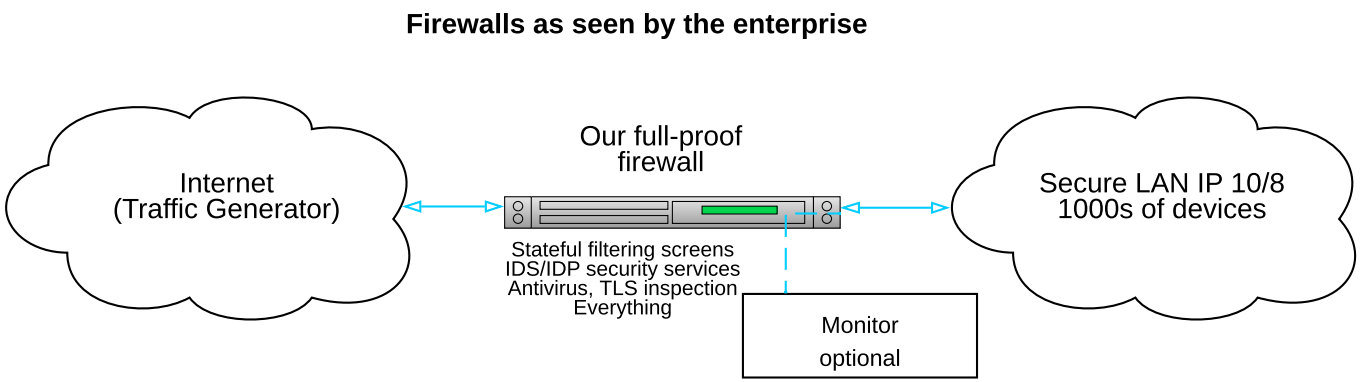
A firewall – in the vendor eyes



"Can your firewall flex in the face of change?
Does it harmonize your network, workload, and application security? Does it protect apps and employees in your hybrid or multicloud environment? Make sure you're covered."

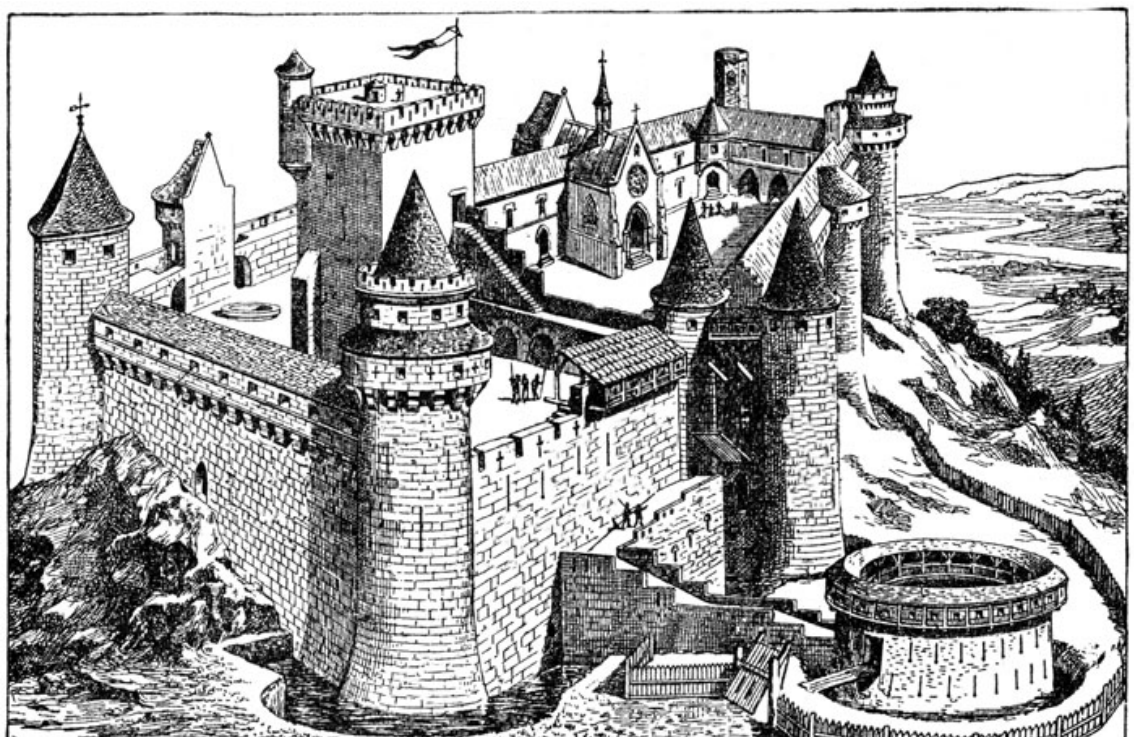
Source: not shown to protect the audience from further marketing speak

A firewall – in the enterprise mindset



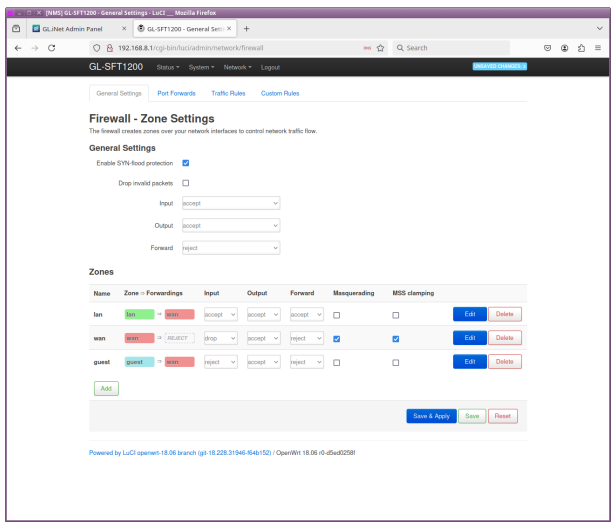
- Even though some vendors suggest they can do everything in a single box, I don't believe them!
- Truth – yes, we can do almost anything in software
- Realization **Your infrastructure is based on multiple components and or devices**

Defense in depth



Picture originally from: http://karenswhimsy.com/public-domain-images

Firewalls



- Firewalls are a great project to look into, there are so many ☺
- I recommend looking into open source firewalls like OPNsense, UFW and OpenBSD and many routers also include firewalls. You can often find images from commercial vendors, that you can try out.
- Picture is from GL-Inet router Opal with OpenWRT/LuCI which is router software for smaller routers

What is a packet filter

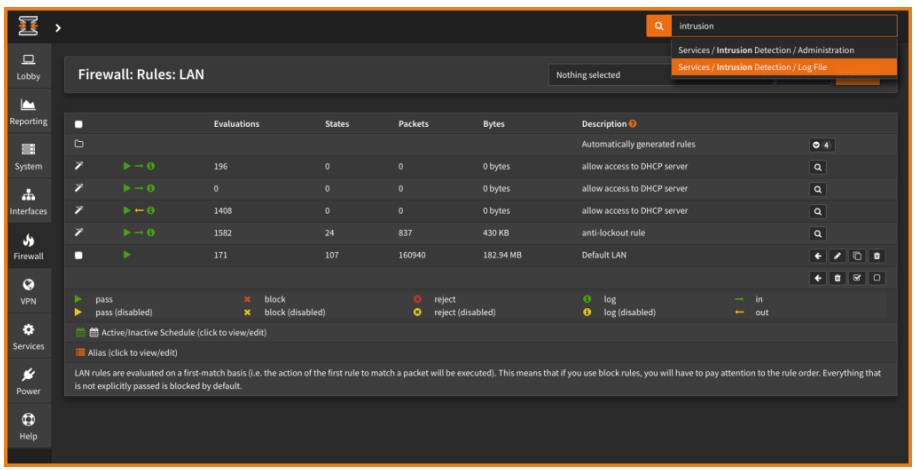


We may want to consider the difference between different types of firewalls/devices:

- Network layer, we often call them packet filters, stateless
- Application level, we often call them, stateful filtering and gateways
- Firewalls are by design a choke point, natural place
  to do network security monitoring!

They are all firewalls – or firewall devices!

Firewall OPNsense GUI based and easy to install



- OPNsense https://opnsense.org/
- Firewall built on FreeBSD with web interface
- Originally thoughts from m0n0wall and later https://www.pfsense.org/

- Danish companies have been using these for many years now

Linux Uncomplicated Firewall (UFW)



Debian / Ubuntu:

```
root@debian01:~# apt install ufw
...
root@debian01:~# ufw allow 22/tcp
Rules updated
Rules updated (v6)
root@debian01:~# ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
root@debian01:~# ufw status numbered
Status: active

     To                         Action      From
     --                         ------      ----
[ 1] 22/tcp                     ALLOW IN    Anywhere
[ 2] 22/tcp (v6)                ALLOW IN    Anywhere (v6)
```

- Extremely easy to use – I recommend and use the (Uncomplicated Firewall) UFW

Sample rules from OpenBSD PF



```
# hosts and networks
router="217.157.20.129"
webserver="217.157.20.131"
homenet=" 192.168.1.0/24, 1.2.3.4/24 "
wlan="10.0.42.0/24"
wireless=wi0
set skip lo0
# things not used
spoofed=" 127.0.0.0/8, 172.16.0.0/12, 10.0.0.0/16, 255.255.255.255/32 "

block in all # default block anything
# egress and ingress filtering - disallow spoofing, and drop spoofed
block in quick from $spoofed to any
block out quick from any to $spoofed

pass in on $wireless proto tcp from { $wlan $homenet } to any port = 22
pass in on $wireless proto tcp from any to $webserver port = 80

pass out
```

Linux TCP SACK PANIC - CVE-2019-11477 et al



Kernel vulnerabilities, CVE-2019-11477, CVE-2019-11478 and CVE-2019-11479

Executive Summary

Three related flaws were found in the Linux kernel's handling of TCP networking. The most severe vulnerability could allow a remote attacker to trigger a kernel panic in systems running the affected software and, as a result, impact the system's availability.

The issues have been assigned multiple CVEs: CVE-2019-11477 is considered an Important severity, whereas CVE-2019-11478 and CVE-2019-11479 are considered a Moderate severity.

The first two are related to the Selective Acknowledgement (SACK) packets combined with Maximum Segment Size (MSS), the third solely with the Maximum Segment Size (MSS).

These issues are corrected either through applying mitigations or kernel patches. Mitigation details and links to RHSA advsories can be found on the RESOLVE tab of this article.

Source: https://access.redhat.com/security/vulnerabilities/tcpsack

Linux TCP IPv4 options UAF - CVE-2026-98432



Kernel vulnerability CVE-2026-98432, present since Linux 4.4 (2016)

Summary

A race condition in the Linux kernel's handling of new IPv4 TCP connections can lead to a use-after-free. IP options stored with a pending connection request are handed to the new socket without being copied. If that socket is destroyed while a retransmitted SYN is still being answered, freed memory is used. IPv6 is not affected.

An unauthenticated remote attacker sends SYNs carrying IPv4 options (Record Route or Timestamp) to a listening port and retries until the race is won. A lost race only fails that one connection attempt, so the attacker can keep trying.

Anthropic demonstrated remote root against servers with a Multipath TCP listener. Go 1.24 and later programs open MPTCP listeners by default.

Fixed in 5.10.272, 5.15.223, 6.1.190, 6.6.159, 6.12.113, 6.18.56 and 7.2.10. As a mitigation, Red Hat suggests filtering incoming IPv4 TCP packets that carry header options.

Source: https://access.redhat.com/security/vulnerabilities/RHSB-2026-016

Rooting devices from far away



Root from kilometers away: RCE in Ubiquiti airMAX antennas

Researchers at Faraday Security (Gastón Aznarez, Federico Kirschbaum, Dan Borgogno) found two unauthenticated remote code execution vulnerabilities in Ubiquiti's outdoor wireless equipment: airMAX and airFiber antennas and UniFi building bridges. These are the devices mounted on rooftops and poles to carry internet, cameras and building links over long distances.

CVEs (both CVSS 3.1 score 8.8, 'AV:A/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H'):

- **CVE-2026-21639**: airMAX AC, airMAX M, airFiber AF60-XG, airFiber AF60
- **CVE-2026-21638**: UBB-XG, UDB-Pro/UDB-Pro-Sector, UBB

Source: https://faradaysec.com/update-your-ubiquiti-equipment-what-we-found-in-airmax-antennas/

Why?!

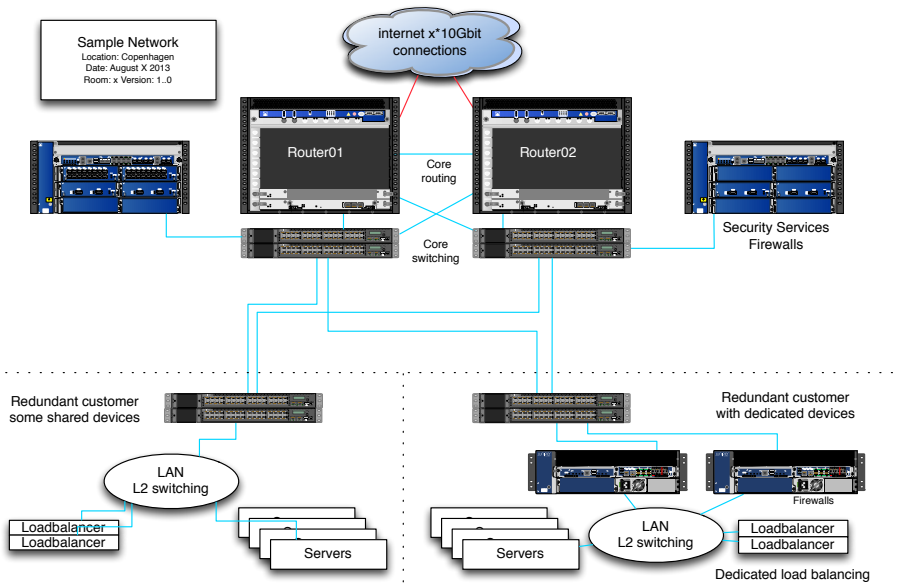


What went wrong

- The proprietary airMAX protocol is carried inside vendor-specific fields of ordinary Wi-Fi management frames. - Parts of that data looked encrypted, but were only obfuscated. - The flaw is a command injection (CWE-77): input from the radio frame reached system command execution without proper sanitisation. - Several affected models ran **internal software around 17 years old, unsupported for close to a decade.**

- Faraday blog post: https://faradaysec.com/update-your-ubiquiti-equipment-what-we-found-in-airmax-antennas/
- Ubiquiti Security Advisory Bulletin 061 (CVE-2026-21639): https://community.ui.com/releases/Security-Advisory-Bulletin-061-061/1e4fe5f8-29c7-4a7d-a518-01b1537983ba
- Ubiquiti Security Advisory Bulletin 060 (CVE-2026-21638): https://community.ui.com/releases/Security-Advisory-Bulletin-060-060/cde18da7-2bc4-41bb-a9cc-48a4a4c479c1
- The talk "Root From Kilometers Away"was presented at Black Hat USA 2026 and DEF CON 34.

Bottlenecks exist, but where



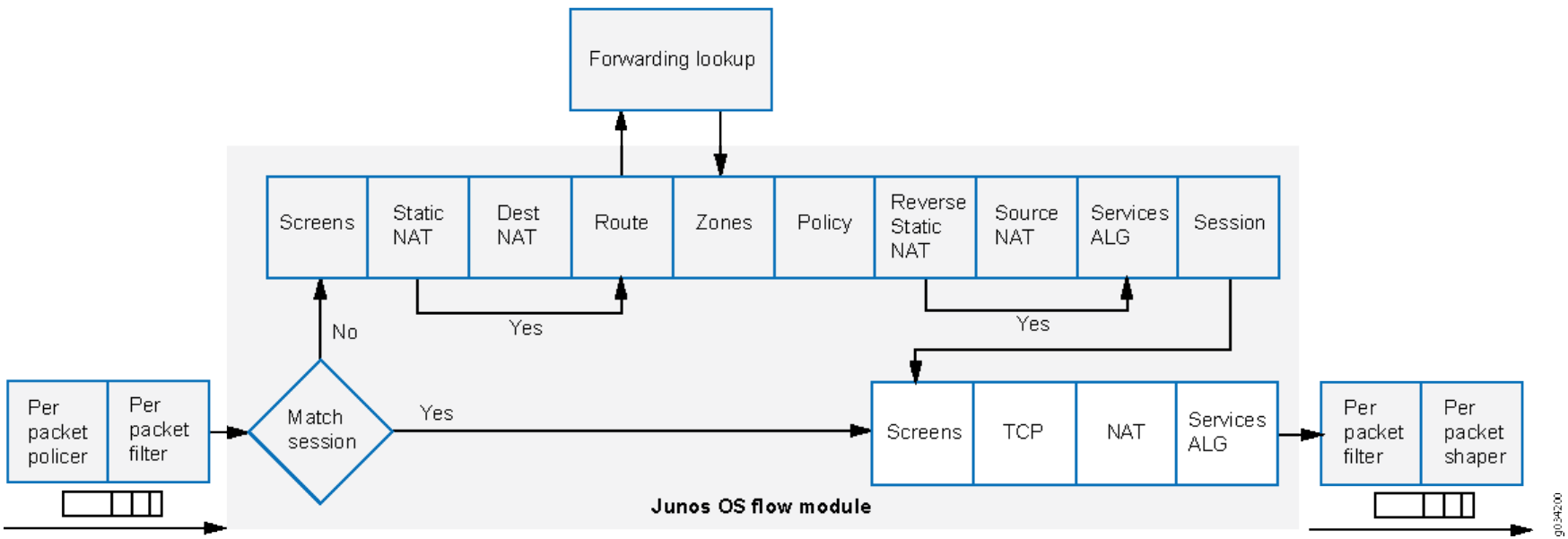
- Lower layer attacks Transport Layer Attacks TCP SYN flood – packet based
- Higher layer attacks like Slowloris and web attacks – keep sessions running
- Protect everything without loosing functionality or creating administrative nightmare

Availability and Network flooding attacks



- **SYN flood** is the most basic and very common on the internet towards 80/tcp and 443/tcp
- **ICMP and UDP flooding** are the next popular targets – more similar ones exist
- Special packets and protocols – anything that can create *load on systems* work
- All of them try to use up some resources
- **Memory space** in specific sections of the **kernel, TCP state, firewalls state, number of concurrent sessions/connections**
- **Interrupt processing** of packets - packets per second (pps)
- **CPU processing** in firewalls, pps
- CPU processing in server software
- **Bandwidth** - megabits per second (mbps)
- Typically source is spoofed or amplification attacks abusing devices on the Internet

Packet processing in firewalls – detailed view



Traffic Processing on SRX Series Devices Overview

https://www.juniper.net/documentation/us/en/software/junos/flow-packet-processing/topics/topic-map/security-srx-devices-processing-overview.html

Simulating DDoS packets



A minimal introduction workshop teaching people how to produce DDoS simulation traffic - usefull for testing their own infrastructures.

We will have a server connected on a switch with multiple 1Gbit port for attackers. Attackers will be connected through 1Gbit ports using USB Ethernet - we have loaners.

Work together to produce enough to take down this server!

WHILE attack is ongoing there will be both the possibility to monitor traffic, monitor port, and decide on changes to prevent the attacks from working.

Common DDoS attack types



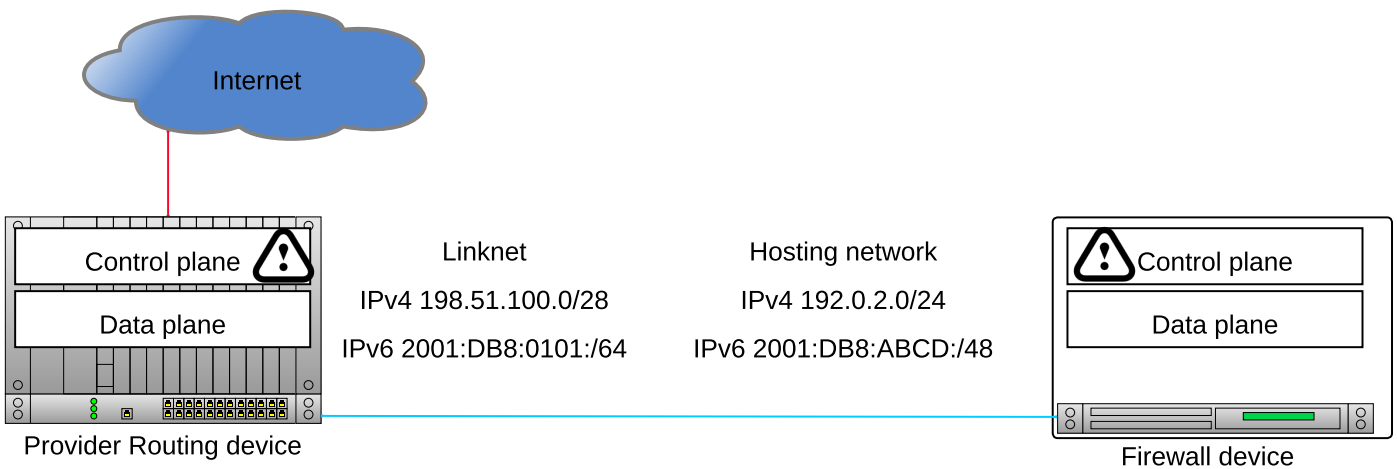
We will work through common attack types, like:

- TCP SYN flooding
- TCP other flooding
- UDP flooding NTP, etc.
- ICMP flooding
- Misc - stranger attacks and illegal combinations of flags etc.

then we will discuss which changes to environment could be implemented.

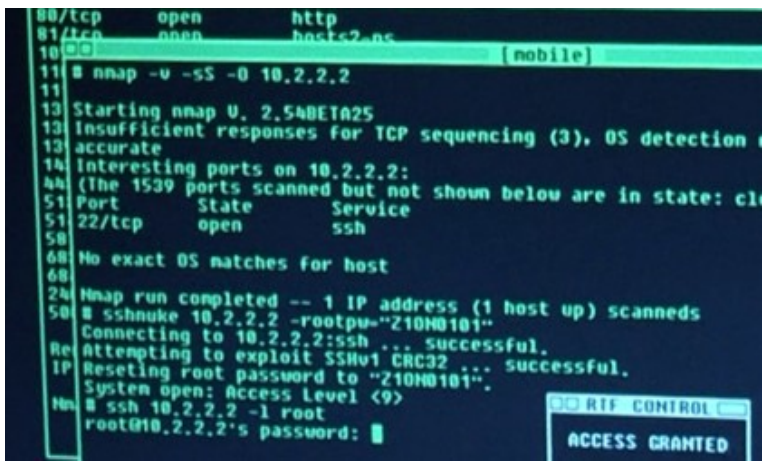
You will go away from this with tools for producing packets, hping3 and some configurations for protecting - PF rules, switch rules, server firewall rules.

Scanning and Attacking – Pressure Points and Scope



- In scope for me is everything that could adversely affect the network
- Typical scope IPv4: Link network /28 or /26 and a hosting network /24 or even /22
- Typical scope IPv6: Link network /64 (bad) or /127 (RFC9099) and a hosting network /48 with subnets

Prepare for the testing



- Portscan the whole linknet and hosting range in IPv4
- Ask about IPv6 ranges in use, specific subnets and IPv6 addresses
  We can guess from traceroute, Nmap test first 100 addresses in each subnet etc. but easier to ask
- Portscan the whole linknet - identify provider devices, hosting network devices, type of device router/firewall
- Also Thank you Fyodor (Gordon Lyon) and contributors for Nmap!

Detailed Scope and Plan



Select a few targets for monitoring and attacks, from the port scans

Best case would be to have:

- Ping ICMP allowed to a provider router and hosting firewall – is the connection alive
- TCP port with service checks, HTTP being attacked and one not being attacked
- UDP port with service check, DNS is a favourite – ask for localhost/127.0.0.1
- Put monitoring on these, a week before testing is nice
- Agree on a day or night for testing, inform participants and system owners

hping3 packet generator



```
usage: hping3 host [options]
  -i  --interval  wait (uX for X microseconds, for example -i u1000)
      --fast      alias for -i u10000 (10 packets for second)
      --faster    alias for -i u1000 (100 packets for second)
      --flood     sent packets as fast as possible. Don't show replies.
...
hping3 is fully scriptable using the TCL language, and packets
can be received and sent via a binary or string representation
describing the packets.
```

- Hping3 packet generator is a very flexible tool to produce simulated DDoS traffic with specific charateristics
- Home page: http://www.hping.org/hping3.html
- Source repository https://github.com/antirez/hping

My primary DDoS testing tool, easy to get specific rate pps

t50 packet generator



```
root@cornerstone03:~# t50 -?
T50 Experimental Mixed Packet Injector Tool 5.4.1
Originally created by Nelson Brito <nbrito@sekure.org>
Maintained by Fernando Mercês <fernando@mentebinaria.com.br>

Usage: T50 <host> [/CIDR] [options]

Common Options:
    --threshold NUM        Threshold of packets to send     (default 1000)
    --flood                This option supersedes the 'threshold'
...
6. Running T50 with '--protocol T50' option, sends ALL protocols sequentially.
root@cornerstone03:~# t50 -? | wc -l
264
```

- T50 packet generator, another high speed packet generator can easily overload most firewalls by producing a randomized traffic with multiple protocols like IPsec, GRE, MIX
  home page: http://t50.sourceforge.net/resources.html

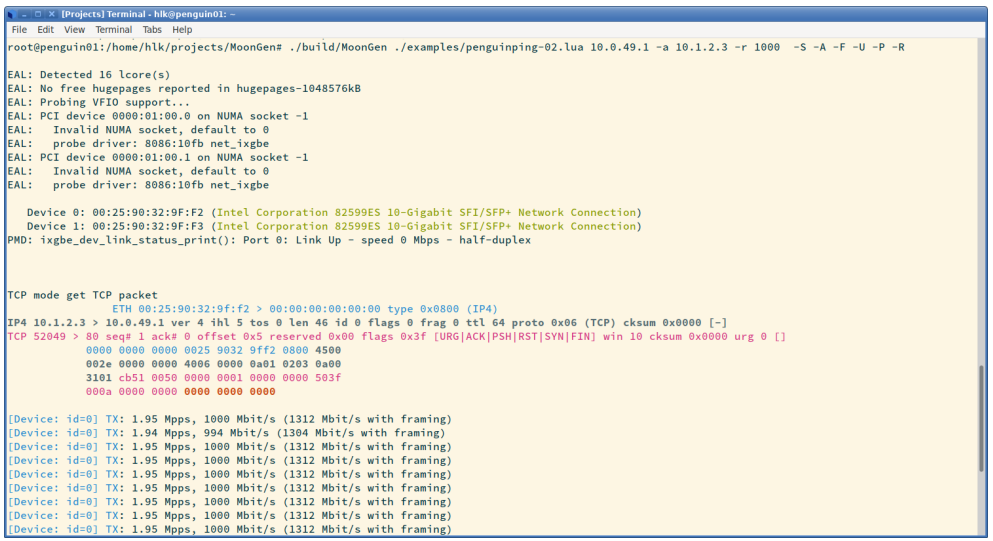
Fast and breaks most firewalls when flooding, easy 800k pps/400Mbps

Enter MoonGen and Dedicated Hardware



- Modern computer with modern CPU and PCIe x8 or better
  I currently use a few cheap Dell devices Precision 3640 Tower / Precision 3240 Compact (not a recommendation)
- Supported card - I am using the old Intel 82599 based 10Gbit cards
- DPDK and software – I use MoonGen https://github.com/emmericp/MoonGen
- A huge thanks to Paul Emmerich emmericp for programming and publishing his works!
- Maybe the easiest way to use DPDK currently – "Craft all packets in user-controlled Lua scripts"

Penguingping packet generator



- PenguinPing packet generator, my high speed packet generator home page: https://penguinping.org
- First versions are only about 230 lines of Lua code and implement basic command line to replace hping3
- Built on top of MoonGen/libmoon https://github.com/emmericp/MoonGen

Extremely fast and allows easy customization

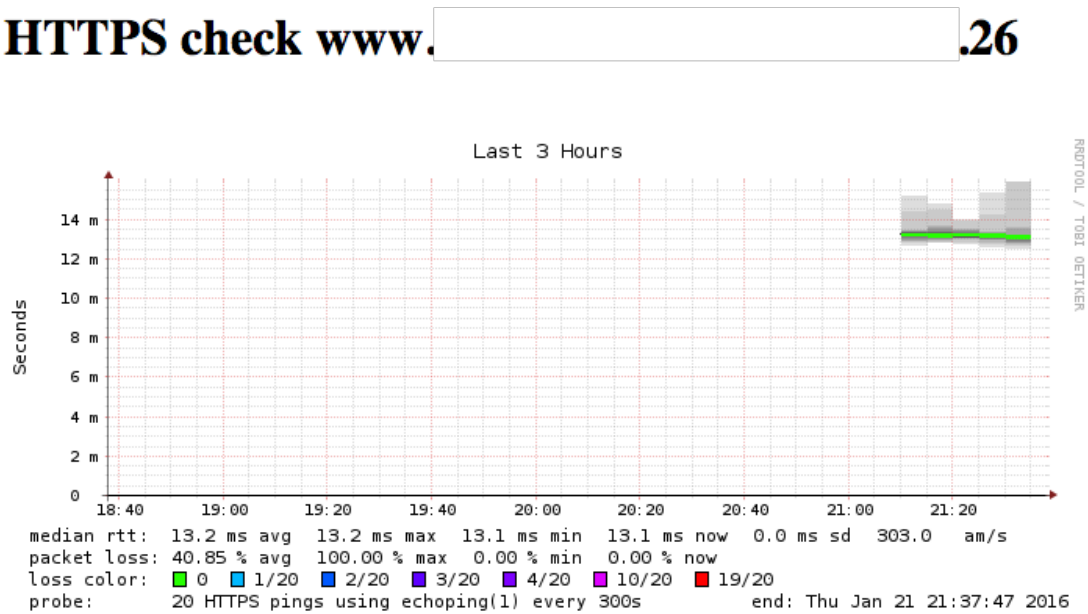
Process: monitor, attack, break, repeat



- Pre-test: Monitoring setup - from multiple points
- Pre-test: Perform full Nmap scan of network and ports
- Start small, run with delays between packets
- Turn up until it breaks, decrease delay - until using `--flood`
- Monitor speed of attack on your router interface pps/bandwidth
- Give it maximum speed
  `hping3 --flood -1` and `hping3 --flood -2`
- Have a common chat with network operators/customer to talk about symptoms and things observed
- Any information resulting from testing is good information

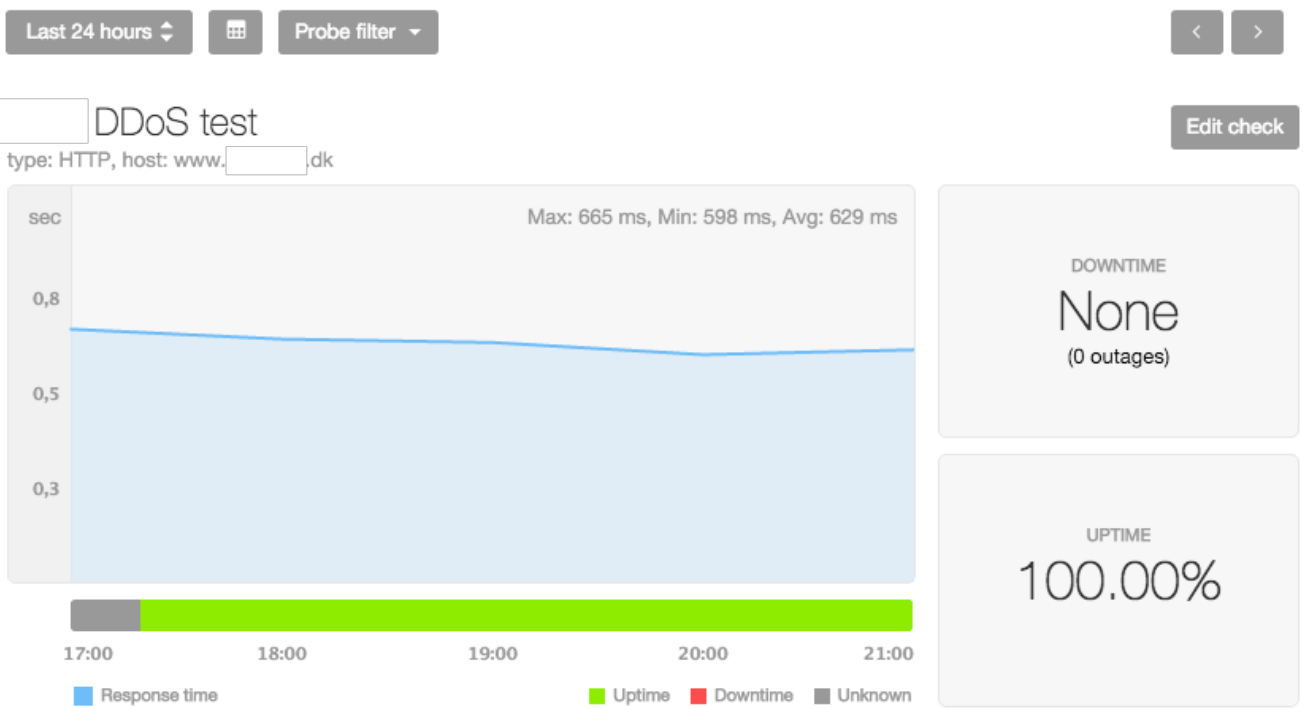
Ohh we lost our VPN into the environment, ohh the fw console is dead

Before testing: Smokeping



Before DDoS testing use Smokeping software

Before testing: Pingdom



Another external monitoring from Pingdom.com

Performing the DDoS test



So lets break this task down into:

- Use Nmap to port scan the network
- Run testing tool `hping3`, `t50` or `penguinping`
- Go through all scenarios before making changes to environment
- Expect things to break, investigate, repeat failed scenarios

- BTW we usually schedule this for night time! There WILL be interruptions

- How do you run the tools, since they all have so many options?
  `man nmap | enscript -o test.ps` result in 54 pages, `hping3` has about 80 options and `t50` has even more

Running full port scan on network



```
# export CUST_NET="192.0.2.0/24"
# nmap -p 1-65535 -A -oA full-scan $CUST_NET
```

Performs a full port scan of the network, all ports

Saves output in "all formats"normal, XML, and grepable formats

Goal is to enumerate the ports that are allowed through the network.

Note: This command is pretty harmless, if something dies, then it is *vulnerable to normal traffic* - and should be fixed!

Running Attacks with hping3



```
# export CUST_IP=192.0.2.1
# date;time hping3 -q -c 1000000  -i u60 -S -p 80  $CUST_IP


# date;time hping3 -q -c 1000000  -i u60 -S -p 80  $CUST_IP
Thu Jan 21 22:37:06 CET 2016
HPING 192.0.2.1 (eth0 192.0.2.1): S set, 40 headers + 0 data bytes

--- 192.0.2.1 hping statistic ---
1000000 packets transmitted, 999996 packets received, 1% packet loss
round-trip min/avg/max = 0.9/7.0/1005.5 ms

real 1m7.438s
user 0m1.200s
sys 0m5.444s
```

Dont forget to do a killall hping3 when done ☺

Recommendations During Test



Run each test for at least 5 minutes, or even 15 minutes
Some attacks require some build-up before resource run out

Take note of any change in response, higher latency, lost probes

If you see a change, then re-test using the same parameters, or a little less first

We want to know the approximate level where it breaks

If you want to change environment, then wait until all scenarios tested

Comparable to real DDoS?



Tools are simple and widely available but are they actually producing same result as high-powered and advanced criminal botnets. We can confirm that the attack delivered in this test is, in fact, producing the traffic patterns very close to criminal attacks in real-life scenarios.

- We can also monitor logs when running a single test-case
- Gain knowledge about supporting infrastructure
- Can your syslog infrastructure handle 800.000 events in $<$ 1 hour?

Running the tools



A basic test would be:

- TCP SYN flooding
- TCP other flags, PUSH-ACK, RST, ACK, FIN
- ICMP flooding
- UDP flooding
- Spoofed packets src=dst=target ☺
- Small fragments
- Bad fragment offset
- Bad checksum
- Be creative
- Mixed packets - like `t50 --protocol T50`
- Perhaps esoteric or unused protocols, GRE, IPSec

Test-cases / Scenarios



The minimal run contains at least these:

- SYN flood: `hping3 -q -c 1000000  -i u60 -S -p 80  $CUST_IP &`
- SYN+ACK: `hping3 -q -c 1000000  -i u60 -S -A -p 80  $CUST_IP &`
- ICMP flood: `hping3 -q -c --flood -1 $CUST_IP &`
- UDP flood: `hping3 -q -c --flood -1 $CUST_IP &`

Vary the speed using the packet interval `-i u60` up/down

Use flooding with caution, runs max speeeeeeeeeeeed ☺

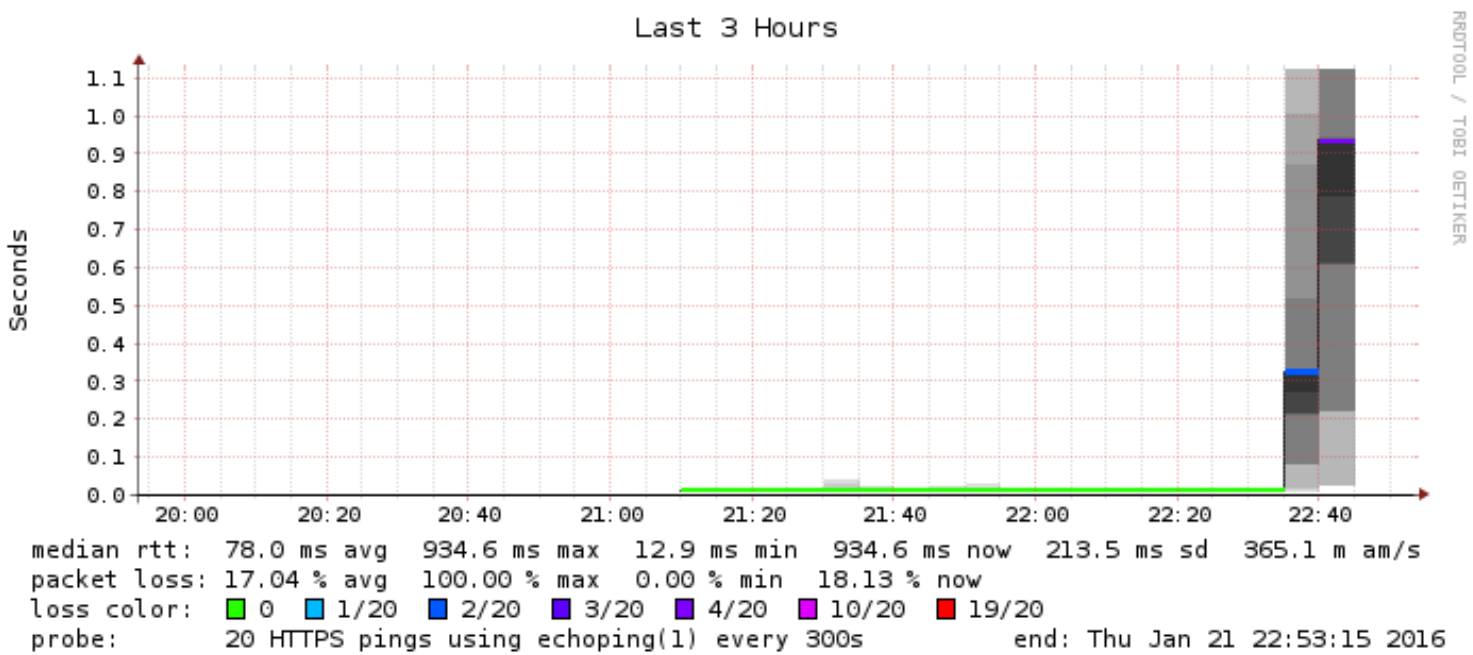
TCP testing use a port which is allowed through the network, often 80/443

Focus on attacks which are hard to block, example TCP SYN must be allowed in

Also if you found devices like routers in front of environment

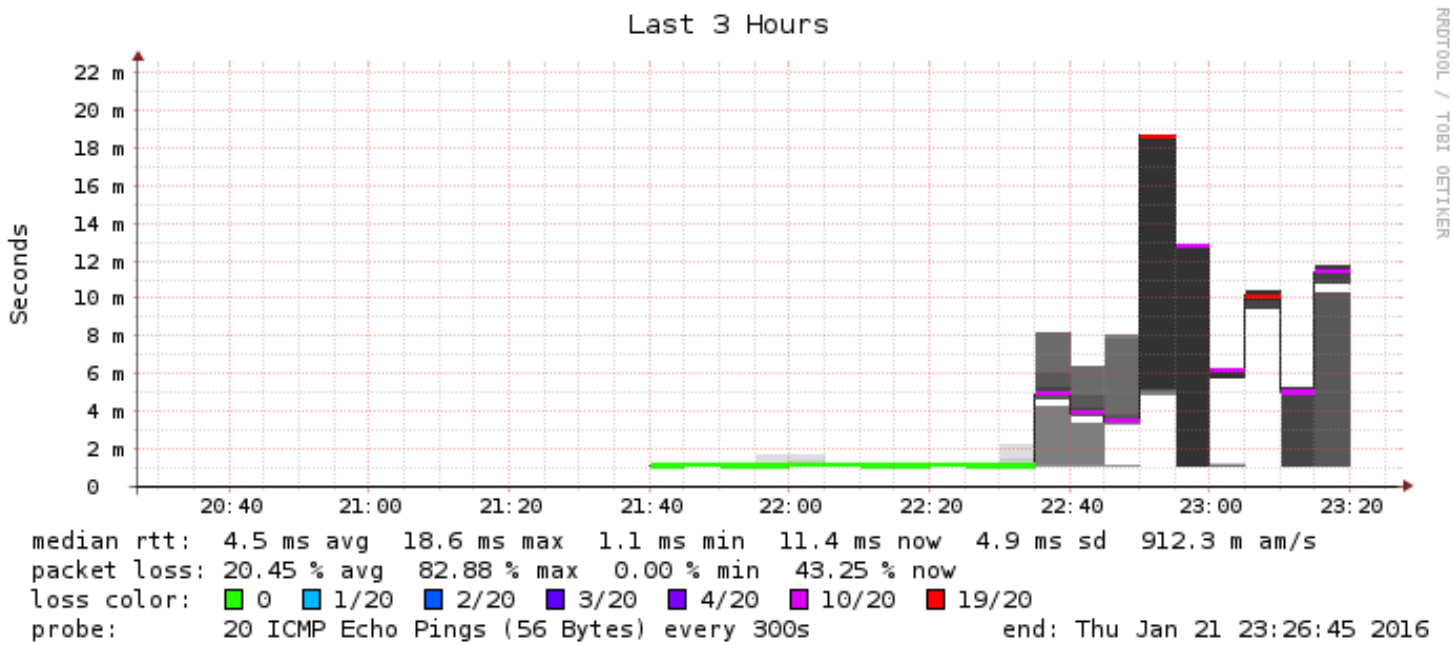
```
hping3 -q -c 1000000  -i u60 -S -p 22 $ROUTER_IP
hping3 -q -c 1000000  -i u60 -S -p 179 $ROUTER_IP
```

Rocky Horror Picture Show - 1



Really does it break from 50.000 pps SYN attack?

Rocky Horror Picture Show - 2

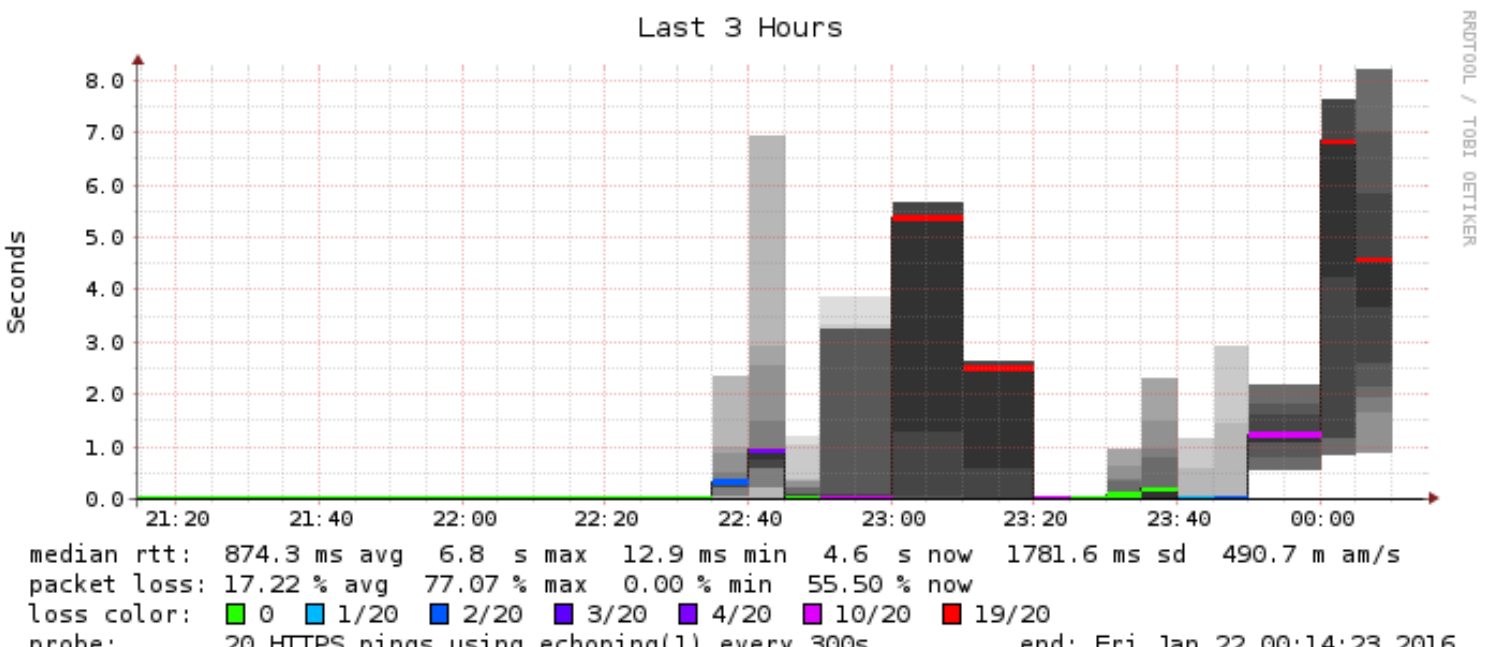


Oh no 500.000 pps UDP attacks work?

Rocky Horror Picture Show - 3



Oh no spoofing attacks work?



Exercise

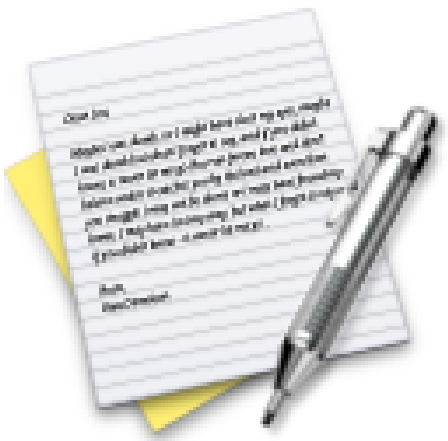


Now lets do the exercise

⚠ Execute nmap TCP and UDP port scan 20 min

which is number 21 in the exercise PDF.

Exercise

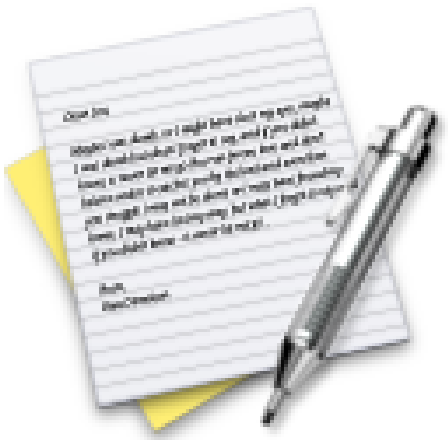


Now lets do the exercise

⚠ Discover active systems ping and port sweep 15 min

which is number 20 in the exercise PDF.

Exercise



Now lets do the exercise

TCP SYN flooding 30min

which is number **33** in the exercise PDF.

Exercise booklet contains some bonus exercises, feel free to try them at home

Improvements seen after testing



Turning off unneeded features - free up resources

Tuning sesions, max sessions src / dst

Tuning firewalls, max sessions in half-open state, enabling services

Tuning network, drop spoofed src from inside net ☺

Tuning network, can follow logs, manage network during attacks

...

And organisation has better understanding of DDoS challenges

Including vendors, firewall consultants, ISPs etc.

After tuning of **existing devices/network** improves results 10-100 times

Conclusion



You really should try testing

Investigate your existing devices
all of them, RTFM, upgrade firmware

Choose which devices does which
part - discard early to free resources
for later devices to dig deeper

And dont forget that DDoS testing is as much a firedrill for the organisation

Stateless firewall filter throw stuff away



```
hlk@MX-CPH-02> show configuration firewall filter all | no-more
/* This is a static sample, perhaps better to use BGP flowspec and RTBH */
term edgeblocker {
    from {
        source-address {
            84.180.xxx.173/32;
            87.245.xxx.171/32;
        }
        destination-address {
            91.102.91.16/28;
        }
        protocol [ tcp udp icmp ];
    }
    then {
        count edge-block;
        discard;
    }
}
```

Hint: can also leave out protocol and then it will match all protocols

Stateless firewall filter limit protocols



```
term limit-icmp {
    from {
        protocol icmp;
    }
    then {
        policer ICMP-100M;
        accept;
    }
}
term limit-udp {
    from {
        protocol udp;
    }
    then {
        policer UDP-1000M;
        accept;
    }
}
```

Routers have extensive Class-of-Service (CoS) tools today

Strict filtering for some servers, still stateless!



```
term some-server-allow {
    from {
        destination-address {
            109.238.xx.0/xx;
        }
        protocol tcp;
        destination-port [ 80 443 ];
    }
    then accept;
}
term some-server-block-unneeded {
    from {
        destination-address {
            109.238.xx.0/xx;
        }
        protocol-except icmp;
    }
    then discard;
}
```

Wut - no UDP, yes UDP service is not used on these servers

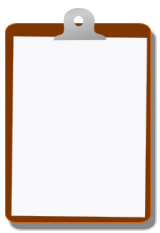
Firewalls - screens, IDS like features



When you know regular traffic you can decide *normal settings*:

```
hlk@srx-kas-05# show security screen ids-option untrust-screen
icmp {
    ping-death;
}
ip {
    source-route-option;
    tear-drop;
}
tcp {    Note: UDP flood setting also exist
    syn-flood {
        alarm-threshold 10024;
        attack-threshold 200;
        source-threshold 10024;
        destination-threshold 2048;
        timeout 20;
    }
    land;
}
```

For Next Time



Think about the subjects from this time, write down questions

Check the plan for chapters to read in the books

Visit web sites and download papers if needed

Retry the exercises to get more confident using the tools